

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра прикладной информатики и теории вероятностей

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 10

дисциплина: Архитектура компьютера —

Студент: Ромеро Гаспар Фернандо Алексей

Группа: НКАбд-02-25

МОСКВА

2025 г.

Оглавление

Цель заботы

Теоретическое введение

2.1 Права доступа к файлам в GNU/Linux

2.2 Системные вызовы NASM для работы с файлами

Выполнения лабораторной работы

Выполнение самостоятельной работы

Выводы

Список литературы

Цель работы

Приобретение навыков написания программ для работы с файлами

Теоретическое введение

2.1 Права доступа к файлам в GNU/Linux

Система разрешений GNU/Linux основана на трёх категориях пользователей: владелец (owner), группа (group) и остальные (others). Для каждой категории определены три разрешения: чтение (r), запись (w) и выполнение (x). Эти разрешения представляются либо в символической нотации (например, `rw xr-xr--`), либо в восьмеричной нотации (например, `755`). Разрешения управляются файловой системой и применяются ядром, определяя, какие пользователи могут читать, изменять или выполнять файлы и каталоги. Разрешения файла можно просмотреть с помощью команды `ls -l`.

Кроме основных разрешений, GNU/Linux включает специальные права доступа: **SUID** (Set User ID — установка идентификатора пользователя), **SGID** (Set Group ID — установка идентификатора группы) и **Sticky Bit** (липкий бит).

- **Право SUID** (обозначается как `s` в позиции выполнения для владельца) позволяет выполнять файл с привилегиями владельца файла, а не пользователя, который его запускает.
- **Право SGID** (`s` в позиции группы) обеспечивает аналогичное поведение для группы. При установке на каталог он заставляет создаваемые внутри файлы наследовать группу родительского каталога.
- **Sticky Bit** (`t` в позиции для остальных) в каталоге ограничивает удаление файлов только их владельцами, что полезно в таких каталогах, как `/tmp`.

Эти права устанавливаются с помощью дополнительных восьмеричных значений (4 для SUID, 2 для SGID, 1 для Sticky Bit) или командой `chmod` (например, `chmod +s файл`).

Разрешения в основном изменяются с помощью команд `chmod` (изменить права доступа), `chown` (изменить владельца) и `chgrp` (изменить группу). Например, `chmod 755 script.sh` предоставляет владельцу полные права, а группе и остальным — право на чтение и выполнение, тогда как `chmod u+x файл` добавляет владельцу право на выполнение.

Безопасность в GNU/Linux в значительной степени зависит от правильной настройки разрешений, применяя принцип минимальных привилегий. Критически важно избегать небезопасных настроек, таких как разрешения `777` (чтение, запись и выполнение для всех) или файлов с ненужным SUID. Инструменты, такие как `umask`, определяют разрешения по умолчанию для новых файлов, а регулярные проверки с помощью команды `find / -perm -4000` помогают находить потенциально опасные файлы с SUID.

Таблица команд для управления разрешениями в GNU/Linux

КОМАНДА	БАЗОВЫЙ СИНТАКСИС	ОПИСАНИЕ	ПРИМЕР	ЭФФЕКТ
LS -L	ls -l [файл/каталог]	Список файлов с деталями, включая права доступа	ls -l документ.tx	Показывает: -rw-r--r-- 1 user group 1234 Jan 28 10:00 документ.txt
CHMOD	chmod [права] файл	Изменяет права доступа файлов/каталогов	chmod 755 script.sh	Присваивает: владельцу: rwx, группе: r-x, остальным: r-x
CHMOD СИМВОЛЬНЫЙ	chmod [u/g/o/a][+/-/=][r/w/x] файл	Изменяет права, используя символьную нотацию	chmod u+x,go-w файл	Добавляет выполнение владельцу, убирает запись у группы и остальных
CHOWN	chown [пользователь]:[группа] файл	Изменяет владельца и/или группу файла	chown карлос:devs app.log	Меняет владельца на карлос и группу на devs
CHGRP	chgrp [группа] файл	Изменяет только группу файла	chgrp www-data сайт.conf	Назначает группу www-data файлу

UMASK	umask [значение]	Определяет права по умолчанию для новых файлов	umask 022	Новые файлы: 644 (rw-r--r--), каталоги: 755 (rwxr-xr-x)
GETFACL	getfacl файл	Показывает расширенные права ACL (списки контроля доступа)	getfacl /var/log/	Выводит базовые права и детализированные ACL
SETFACL	setfacl -m u:[пользователь]:[права] файл	Устанавливает определённые права ACL	setfacl -m u:анна:rw-отчёт.txt	Даёт анне права на чтение и запись
STAT	stat файл	Показывает детальную информацию, включая права в восьмеричной системе	stat script.py	Показывает: Access: (0754/-rwxr-xr--)
FIND ПРАВАМИ	find /путь [режим] -perm	Ищет файлы с определённым и правами	find /home -perm 777	Находит файлы с правами 777 (rwxrwxrwx)
CHATTR	chattr [+/-][i/a] файл	Изменяет специальные атрибуты файловой системы	chattr +i /etc/config.conf	Делает файл неизменяемым (нельзя изменять/удалять)
LSATTR	lsattr файл	Списывает специальные атрибуты файлов	lsattr /etc/hosts	Показывает атрибуты, например i (immutable), a (только добавление)

Восьмеричная нотация:

- 4 = чтение (r)
- 2 = запись (w)
- 1 = выполнение (x)
- Пример: 755 = 4+2+1(владелец) 4+0+1(группа) 4+0+1(остальные) = rwxr-xr-x

Специальные права:

- SUID (Set User ID): `chmod u+s` или `chmod 4755`
- SGID (Set Group ID): `chmod g+s` или `chmod 2755`
- Sticky Bit: `chmod +t` или `chmod 1777`

Символы в `ls -l`:

- Первая позиция: - (файл), d (каталог), l (ссылка)
- Следующие 9 позиций: rwxrwxrwx (владелец|группа|остальные)

Рекомендации по безопасности:

- Избегать 777 для критических файлов
- Использовать `chown root:root` и `chmod 755` для системных исполняемых файлов

2.2 Системные вызовы NASM для работы с файлами

В NASM для Linux x86-64 системные вызовы представляют собой прямые обращения к ядру для выполнения низкоуровневых операций, включая работу с файлами. Они осуществляются с помощью инструкции ``syscall``, причём параметры передаются через регистры согласно соглашению: ``rax`` (номер системного вызова), ``rdi`` (первый аргумент), ``rsi`` (второй), ``rdx`` (третий), ``r10`` (четвёртый), ``r8`` (пятый) и ``r9`` (шестой). Для работы с файлами ключевыми системными вызовами являются: ``open`` (2), ``read`` (0), ``write`` (1), ``close`` (3), ``lseek`` (8) и ``stat`` (4). Номера системных вызовов определены в файле ``/usr/include/asm/unistd_64.h``.

Открытие файлов осуществляется с помощью вызова `'open'` со следующими аргументами: `'rdi'` (указатель на имя файла), `'rsi'` (флаги доступа) и `'rdx'` (права доступа, если файл создаётся). Возвращает дескриптор файла (file descriptor) в `'rax'` или отрицательное значение в случае ошибки.

Чтение и запись используют вызовы `'read'/'write'` с аргументами: `'rdi'` (дескриптор), `'rsi'` (буфер), `'rdx'` (количество байт для чтения/записи), возвращая количество переданных байт.

Крайне важно закрывать дескрипторы с помощью `'close'`, чтобы избежать утечек ресурсов.

Для продвинутых операций, `'lseek'` (8) перемещает указатель файла с аргументами: `'rdi'` (дескриптор), `'rsi'` (смещение), `'rdx'` (`'whence': 'SEEK_SET'=0, 'SEEK_CUR'=1, 'SEEK_END'=2`). `'stat'` (4) получает метаданные файла в буфер.

Обработка ошибок проверяет, является ли значение в `'rax'` отрицательным (код ошибки в дополнительном коде, например, `-2 = 'ENOENT'` — «файл не существует»). Ошибки должны быть преобразованы в положительное число с помощью `'neg rax'` и корректно обработаны.

Системные вызовы по умолчанию синхронны и блокирующие; для асинхронного ввода-вывода требуются механизмы, такие как `'io_uring'` или `'epoll'`.

Крайне важно выровнять стек по 16-байтной границе перед системным вызовом и сохранить неиспользуемые регистры.

СИСТЕМН ЫЙ	НОМ ЕР	ПАРАМЕ ТРЫ	ОПИСАНИЕ	РАСПРОСТРАНЁ ННЫЕ
---------------	-----------	---------------	----------	----------------------

ВЫЗОВ	(RAX)			ЗНАЧЕНИЯ
OPEN	2	rdi: путь, rsi: флаги, rdx: режим	Открывает/создаёт файл	флаги: O_RDONLY=0, O_WRONLY=1, O_RDWR=2, O_CREAT=64, O_APPEND=1024, O_TRUNC=512
READ	0	rdi: дескриптор, rsi: буфер, rdx: количество	Читает данные	Возвращает прочитанные байты или -ошибка
WRITE	1	rdi: дескриптор, rsi: буфер, rdx: количество	Записывает данные	Возвращает записанные байты или -ошибка
CLOSE	3	rdi: дескриптор	Закрывает дескриптор	Возвращает 0 (успех) или -ошибка
LSEEK	8	rdi: дескриптор, rsi: смещение, rdx: откуда	Перемещает указатель позиции	whence: SEEK_SET=0 (начало), SEEK_CUR=1 (текущая), SEEK_END=2 (конец)
STAT	4	rdi: путь, statbuf	Получает метаданные файла	statbuf: структура 144 байта
FSTAT	5	rdi: дескриптор, rsi: statbuf	Метаданные по дескриптору	Аналогично stat, но через дескриптор
TRUNCATE	76	rdi: путь, rdx: длина	Обрезает файл до длины	длина: новый размер в байтах
FTRUNCATE	77	rdi: дескриптор,	Обрезает по дескриптору	Аналогично truncate

UNLINK	87	rsi: длина rdi: путь	Удаляет файл	Возвращает 0 или - ошибка
RENAME	82	rdi: старый_путь, rsi: новый_путь	Переименовывает/пере мещает файл,	Возвращает 0 или - ошибка

Флаги открытия (open flags):

- O_RDONLY — только чтение
- O_WRONLY — только запись
- O_RDWR — чтение и запись
- O_CREAT — создать если не существует
- O_TRUNC — обрезать до нулевой длины
- O_APPEND — добавлять в конец

Выполнения лабораторной работы

Создайте рабочую директорию и необходимые файлы с помощью команд `mkdir` и `touch`.

Затем вставьте текст программы из листинга 10.1 в файл `lab10-1.asm`.

Программа запросит у пользователя строку и запишет её в файл `readme.txt`.

Скомпилируйте файл и запустите его.

```
fernando@ubuntu25-10:~/work/arch-pc/lab10$ nasm -f elf lab10-1.asm
fernando@ubuntu25-10:~/work/arch-pc/lab10$ ld -m elf_i386 -o lab10-1 lab10-1.o
fernando@ubuntu25-10:~/work/arch-pc/lab10$ ./lab10-1
Введите строку для записи в файл: Hello, lab 10
fernando@ubuntu25-10:~/work/arch-pc/lab10$ ls -l readme-1.txt
-rw-rw-r-- 1 fernando fernando 0 Jan 31 01:29 readme-1.txt
fernando@ubuntu25-10:~/work/arch-pc/lab10$ cat readme-1.txt
```

С помощью команды `chmod` им удастся снять права на выполнение с файла `lab10-1` (двоичного файла) и попытаться его запустить.

```
fernando@ubuntu25-10:~/work/arch-pc/lab10$ chmod u-x lab10-1
fernando@ubuntu25-10:~/work/arch-pc/lab10$ ls -l lab10-1
-rw-rwxr-x 1 fernando fernando 9164 Jan 31 01:38 lab10-1
fernando@ubuntu25-10:~/work/arch-pc/lab10$ ./lab10-1
bash: ./lab10-1: Permission denied
```

Операционная система запретила выполнение файла, поскольку у текущего пользователя (владельца) отсутствует бит разрешения на выполнение (x). Вы добавили права на выполнение к исходному текстовому файлу `lab10-1.asm`, и я попытался его запустить.

```
fernando@ubuntu25-10:~/work/arch-pc/lab10$ chmod u+x lab10-1.asm
fernando@ubuntu25-10:~/work/arch-pc/lab10$ ./lab10-1.asm
./lab10-1.asm: line 1: fg: no job control
./lab10-1.asm: line 3: SECTION: command not found
./lab10-1.asm: line 4: filename: command not found
./lab10-1.asm: line 5: msg: command not found
./lab10-1.asm: line 7: SECTION: command not found
./lab10-1.asm: line 8: contents: command not found
./lab10-1.asm: line 10: SECTION: command not found
./lab10-1.asm: line 11: global: command not found
./lab10-1.asm: line 12: _start:: command not found
./lab10-1.asm: line 14: syntax error near unexpected token `;'
./lab10-1.asm: line 14: `'; --- Печать сообщения `msg`'
```

Оболочка попыталась выполнить текстовый файл как скрипт. Поскольку синтаксис ассемблера отличается от синтаксиса команд `bash`, возникли ошибки. Исходный код необходимо сначала скомпилировать.

Согласно таблице параметров (параметр 11), установите права доступа следующим образом:

1. Для `readme-1.txt` (символическое представление): `--x r-- -w-`
 - 1.1. Владелец: Только выполнение (`--x`)
 - 1.2. Группа: Только чтение (`r--`)
 - 1.3. Другие: Только запись (`-w-`)
2. Для `readme-2.txt` (двоичное представление): `000 100 111`
 - 2.1. `000 = 0 (---)` - нет прав доступа
 - 2.2. `100 = 4 (r--)` - Только чтение
 - 2.3. `111 = 7 (rwx)` - Чтение, запись и выполнение
 - 2.4. Результирующее восьмеричное значение: `047`

```
fernando@ubuntu25-10:~/work/arch-pc/lab10$ chmod u--x,g=r--,o=-w- readme-1.txt
fernando@ubuntu25-10:~/work/arch-pc/lab10$ chmod 047 readme-2.txt
fernando@ubuntu25-10:~/work/arch-pc/lab10$ ls -l readme-*.txt
----r----- 1 fernando fernando 0 Jan 31 01:29 readme-1.txt
----r--rwx 1 fernando fernando 0 Jan 31 01:29 readme-2.txt
```

Выполнение самостоятельной работы

1. Напишите программу работающую по следующему алгоритму:

- Вывод приглашения “Как Вас зовут?”
- ввести с клавиатуры свои фамилию и имя
- создать файл с именем `name.txt`

- записать в файл сообщение “Меня зовут”
- дописать в файл строку введенную с клавиатуры
- закрыть файл

```
fernando@ubuntu25-10:~/work/arch-pc/lab10$ touch lab10-2.asm
fernando@ubuntu25-10:~/work/arch-pc/lab10$ nano lab10-2.asm
fernando@ubuntu25-10:~/work/arch-pc/lab10$ nasm -f elf lab10-2.asm -o lab10-2.o
fernando@ubuntu25-10:~/work/arch-pc/lab10$ ld -m elf_i386 -o lab10-2 lab10-2.o
fernando@ubuntu25-10:~/work/arch-pc/lab10$ ./lab10-2
Как Вас зовут? Фернандо Алексей
fernando@ubuntu25-10:~/work/arch-pc/lab10$ ls -l name.txt
-rw-r--r-- 1 fernando fernando 52 Jan 31 22:42 name.txt
fernando@ubuntu25-10:~/work/arch-pc/lab10$ cat name.txt
Меня зовут Фернандо Алексей
```

Выводы

В ходе лабораторной работы вы научились работать с файлами в Linux, используя язык ассемблера NASM. Вы изучили системные вызовы `sys_creat`, `sys_open`, `sys_write` и `sys_close` и научились использовать файловые дескрипторы. Вы также укрепили свои навыки управления правами доступа к файлам с помощью команды `chmod`, как в символьном, так и в числовом режимах.

Список литературы

1. Linux Documentation Project. (2023). *Permissions*. https://tldp.org/LDP/intro-linux/html/sect_03_04.html
2. Red Hat Enterprise Linux Documentation. (2024). *Managing file permissions*. https://access.redhat.com/documentation/en-us/red_hat_enterprise_linux/9/html/managing_files_and_directories/managing-file-permissions_managing-files-and-directories
3. Ubuntu Documentation. (2024). *File permissions*. <https://help.ubuntu.com/community/FilePermissions>
4. **Основной источник:** Linux man-pages (2024). *chmod(1), chown(1), umask(2)*. <https://man7.org/linux/man-pages/>
5. GeeksforGeeks. (2023). *System Calls in Operating System*. <https://www.geeksforgeeks.org/system-calls-in-operating-system/>
6. Linux Kernel Documentation. (2024). **System call table for x86-64**. https://github.com/torvalds/linux/blob/master/arch/x86/entry/syscalls/syscall_64.tbl
7. Stack Overflow. (2023). **How to open a file in assembly x86-64**. <https://stackoverflow.com/questions/74190083/how-to-open-a-file-in-assembly-x86-64>
8. Syscall Table. (2024). *Linux x86_64 syscall table*. <https://syscalls64.paolostivanin.com/>
- 9.